

SQL

SQL Tutorial

This **SQL tutorial** is carefully designed by experts to guide learners of all levels, from beginners to professionals, in mastering both basic and advanced SQL concepts. It covers how to create, read, update, and delete data, organize databases, and write efficient queries to extract meaningful data.

This **SQL tutorial** is carefully designed by experts to guide learners of all levels, from beginners to professionals, in mastering both basic and advanced SQL concepts. It covers how to create, read, update, and delete data, organize databases, and write efficient queries to extract meaningful data.

What is SQL?

SQL (Structured Query Language) is a special language used to manage and manipulate data stored in databases. With SQL, you can perform operations like inserting, updating, deleting records, and creating or modifying database tables, views, and more.

SQL is **not a database system** itself - it's a language that works with database management systems (DBMS) such as MySQL, Oracle, PostgreSQL, SQL Server, MongoDB, DB2, and others. To run SQL queries, you need one of these DBMS installed on your system.

SQL is pronounced as "S-Q-L" or sometimes "See-Quell", it is mainly used to handle structured data, which is stored in tables in relational database management systems (RDBMS). It allows you to easily create databases, modify tables, and access or update data efficiently.

Importance of SQL in Data Management and Careers

SQL became the **standard language** for databases in 1986 (ANSI) and 1987 (ISO). It is an essential skill for anyone aiming for a career in data science or working with large datasets. Big companies like Facebook, Instagram, and LinkedIn rely on SQL to manage their data in the back-end.

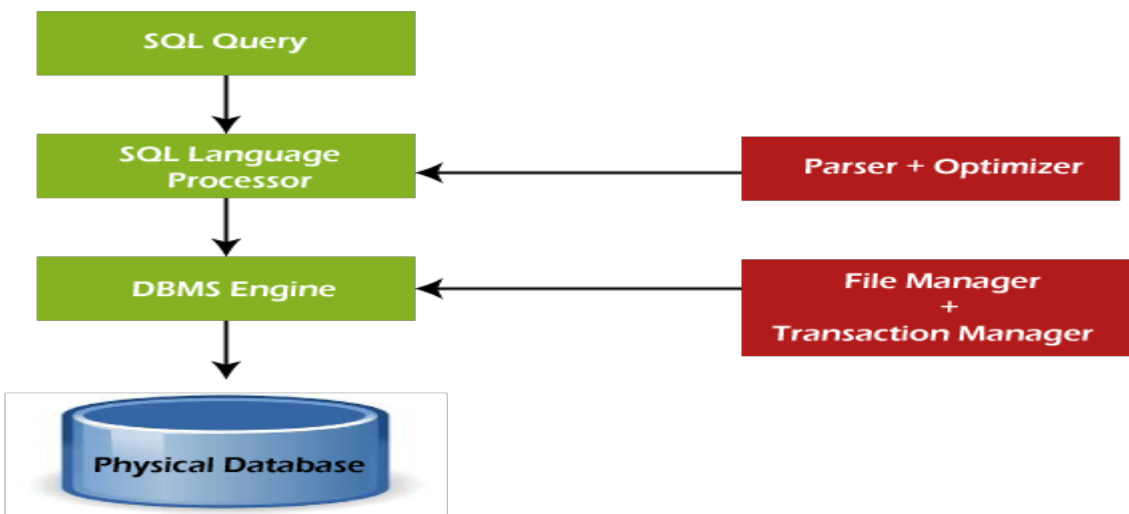
How SQL Works?

When we are executing the command of SQL on any Relational database management system, then the system automatically finds the best routine to carry out our request, and the SQL engine determines how to interpret that particular command.

Structured Query Language contains the following four components in its process:

- Query Dispatcher
- Optimization Engines
- Classic Query Engine
- SQL Query Engine, etc.

A classic query engine allows data professionals and users to maintain non-SQL queries. The architecture of SQL is shown in the following diagram:



Why to Learn SQL?

Nowadays, SQL is widely used in data science and analytics. Following are the reasons which explain why it is widely used:

- The basic use of SQL for data professionals and SQL users is to insert, update, and delete the data from the relational database.
- SQL allows the data professionals and users to retrieve the data from the relational database management systems.
- It also helps them to describe the structured data.
- It allows SQL users to create, drop, and manipulate the database and its tables.
- It also helps in creating the view, stored procedure, and functions in the relational database.
- It allows you to define the data and modify that stored data in the relational database.
- It also allows SQL users to set the permissions or constraints on table columns, views, and stored procedures.

Advantages of SQL

SQL provides various advantages which make it more popular in the field of data science. It is a perfect query language which allows data professionals and users to communicate with the database.

1. No programming needed

SQL does not require a large number of coding lines for managing the database systems. We can easily access and maintain the database by using simple SQL syntactical rules. These simple rules make the SQL user-friendly.

2. High-Speed Query Processing

A large amount of data is accessed quickly and efficiently from the database by using SQL queries. Insertion, deletion, and updation operations on data are also performed in less time.

3. Standardized Language

SQL follows the long-established standards of ISO and ANSI, which offer a uniform platform across the globe to all its users.

4. Portability

The structured query language can be easily used in desktop computers, laptops, tablets, and even smartphones. It can also be used with other applications according to the user's requirements.

5. Interactive language

We can easily learn and understand the SQL language. We can also use this language for communicating with the database because it is a simple query language. This language is also used for receiving the answers to complex queries in a few seconds.

6. More than one Data View

The SQL language also helps in making the multiple views of the database structure for the different database users.

Disadvantages of SQL

1. Cost

The operation cost of some SQL versions is high. That's why some programmers cannot use the Structured Query Language.

2. Interface is Complex

Another big disadvantage is that the interface of Structured query language is difficult, which makes it difficult for SQL users to use and manage it.

3. Partial Database control

The business rules are hidden. So, the data professionals and users who are using this query language cannot have full database control.

SQL Syntax

When you want to do some operations on the data in the database, then you must have to write the query in the predefined syntax of SQL.

The syntax of the structured query language is a unique set of rules and guidelines, which is not case-sensitive. Its Syntax is defined and maintained by the ISO and ANSI standards.

- You can write the keywords of SQL in both uppercase and lowercase, but writing the SQL keywords in uppercase improves the readability of the SQL query.
- SQL statements or syntax are dependent on text lines. We can place a single SQL statement on one or multiple text lines.
- You can perform most of the action in a database with SQL statements.
- SQL syntax depends on relational algebra and tuple relational calculus.

SQL Statement

SQL statements tell the database what operation you want to perform on the structured data and what information you would like to access from the database.

The statements of SQL are very simple and easy to use and understand. They are like plain English but with a particular syntax.

Simple Example of SQL statement:

```
SELECT "column_name" FROM "table_name";
```

Each SQL statement begins with any of the SQL keywords and ends with the semicolon (;). The semicolon is used in the SQL for separating the multiple Sql statements which are going to execute in the same call. In this SQL tutorial, we will use the semicolon (;) at the end of each SQL query or statement.

SQL Create Database

In SQL, the 'Create Database' statement is a first step for storing the structured data in the database.

The database developers and the users use this statement in SQL for creating the new database in the database systems. It creates the database with the name which has been specified in the Create Database statement.

Syntax of Create Database statement in SQL

```
CREATE DATABASE Database_Name;
```

In this syntax, **Database_Name** specifies the name of the database which we want to create in the system. We have to type the database name in query just after the 'Create Database' keyword.

Following are the most important points which are required to learn while creating a database:

- The database we want to create should be a simple and unique name, which can be easily identified.
- Database name should be no more than 128 characters.

Syntax of Create Database statement in MySQL

The same command is used in MySQL to create the new database for storing the structured data.

```
CREATE DATABASE Database_Name;
```

Syntax of Create Database in Oracle

There is no need to create the database in Oracle systems. In the Oracle database, we can directly create the database tables.

Examples of Create Database statement in SQL

In this article, we took the following two examples which will help how to run and perform the Create Database query in SQL:

Example 1:

This example creates the **Student** database. To create the Student database, you have to type the following command in Structured Query Language:

```
CREATE DATABASE Student ;
```

When this query is executed successfully, then it will show the following output:

Database created successfully

You can also verify that your database is created in SQL or not by using the following query:

```
SHOW DATABASE ;
```

SQL does not allow developers to create the database with the existing database name. Suppose if you want to create another Student database in the same database system, then the Create Database statement will show the following error in the output:

Can't create database 'Student'; database exists

So, firstly you have to delete the existing database by using the Drop Statement. You can also replace the existing database with the help of Replace keyword.

If you want to replace the existing Student database, then you have to type the following SQL query:

```
CREATE OR REPLACE DATABASE Student ;
```

Example 2:

Suppose, we want to create the Employee database in the system.

Firstly, we have to type the following command in Structured Query Language:

```
CREATE DATABASE Employee ;
```

When this query is executed successfully, then it will show the following output:

Database created successfully

You can also check that your database is created in SQL by typing the following query:

```
SHOW DATABASE ;
```

We know that SQL does not allow developers to create the database with the existing database name.

Suppose, we want to create another Employee database in the same database system, firstly, we have to delete the existing database using a drop statement, or we have to replace the existing Employee database with the help of the 'replace' keyword.

To replace the existing Employee database with a new Employee database, we have to type the following query in SQL:

```
CREATE OR REPLACE DATABASE Employee;
```

In sqlite3

```
sqlite3 student.db
```

```
SQLite version 3.31.1 2020-01-27 19:55:54
```

```
Enter ".help" for usage hints.
```

```
sqlite> create table students (id INTEGER PRIMARY KEY, name TEXT NOT NULL, age INTEGER);
```

```
sqlite> insert into students (name, age) VALUES ('Ram', 100);
```

```
sqlite> insert into students (name, age) VALUES ('Shiv', 101);
```

```
sqlite> SELECT * from students;
```

```
1|Ram|100
```

```
2|Shiv|101
```

```
sqlite> .quit
```

SQL SELECT Database

Suppose database users and administrators want to perform some operations on tables, views, and indexes on the specific existing database in SQL. Firstly, they have to select the database on which they want to run the database queries.

Any database user and administrator can easily select the particular database from the current database server using the **USE** statement in SQL.

Syntax of USE statement in SQL

```
USE database_name;
```

In this syntax, we have to define the name of the database after the **USE** keyword and the name of the database must be unique.

Syntax of USE statement in MySQL

```
USE database_name;
```

Syntax of USE statement in Oracle

There is no need to select the database in Oracle.

Examples of USE statement in SQL

In this article, we have taken the following three examples which will help you how to run and perform USE statement in SQL:

Example 1: Suppose, you want to work with the **Hospital** database. For this firstly, you have to check that if the Hospital database exists on the current database server or not by using the following query:

```
SHOW DATABASES;
```

If the Hospital database is shown in the output, then you have to execute the following query to select the Hospital database:

```
USE Hospital;
```

Example 2: Suppose, you want to work with another College database in SQL. For this firstly, you have to check that the College database exists on the current database server or not by using the following query:

```
SHOW DATABASES;
```

If the College database is shown in the result, then you have to execute the following query to select the College database:

```
USE College;
```

Example 3: Suppose you want to work with another School database in SQL. For this firstly, you have to check that the School database exists on the current database server or not by using the following query:

```
SHOW DATABASES;
```

If the School database is shown in the result, then you have to execute the following query to select the School database:

```
USE School;
```

In sqlite3

```
sqlite3 student.db
```

```
SQLite version 3.31.1 2020-01-27 19:55:54
```

```
Enter ".help" for usage hints.
```

```
sqlite>
```

SQL RENAME Database

In some situations, database users and administrators want to change the name of the database for some technical reasons. So, the **Rename Database** statement in SQL is used to change the name of the existing database.

Sometimes, the Rename Database statement is used because the developers think that the original name is not more relevant to the data of the database, or they want to give a temporary name to that database.

Syntax of Rename Database in SQL

```
ALTER DATABASE old_database_name MODIFY NAME =  
new_database_name;
```

```
EXEC sp_renamedb'old_database_name' , 'new_database_name'
```

Syntax of Rename Database in MySQL

```
RENAME DATABASE old_database_name TO new_database_name;
```

This syntax is used when we want to change the name of the database in MySQL.

Examples of Rename Database in SQL

In this article, we have taken the following two examples which will help you how to run and perform the Rename Database query in SQL:

Example 1:

Suppose we want to rename the Student Database. For this, we have to type the following query in SQL:

```
ALTER DATABASE Student MODIFY NAME = College;
```

This query will change the name of the database from Student to College. To run this query, we must ensure that the database Student exists in the current database server. If not, then it will show an error in the output.

Example 2:

Suppose we want to rename the Department Database. For this, we have to type the following query in SQL:

```
ALTER DATABASE Department MODIFY NAME = Company ;
```

This query changes the name of the database from Department to Company. To run this query, we must ensure that the database Department exists in the current database server. If not, then it will show an error in the output.

In sqlite3

```
mv student.db class.db
```

```
sqlite3 class.db
```

```
SQLite version 3.31.1 2020-01-27 19:55:54
```

```
Enter ".help" for usage hints.
```

```
sqlite> SELECT * from students;
```

```
1|Ram|100
```

```
2|Shiv|101
```

SQL DROP Database

The SQL Drop Database statement deletes the existing database permanently from the database system. This statement deletes all the views and tables if stored in the database, so be careful while using this query in SQL.

Following are the most important points which are required to learn before removing the database from the database system:

- This statement deletes all the data from the database. If you want to restore the deleted data in the future, you should keep the backup of data of that database which you want to delete.
- Another most important point is that you cannot delete that database from the system which is currently in use by another database user. If you do so, then the drop statement shows the following error on screen:
Cannot drop database "name_of_the_database" because it is currently in use.

Syntax of Drop Database Statement in SQL

```
DROP DATABASE Database_Name;
```

In this SQL syntax, we have to specify the name of that database which we want to delete permanently from the database system. We have to write the name of the database after the DROP DATABASE keyword in every example.

We can also delete multiple databases easily by using the single DROP syntax:

```
DROP DATABASE Database_Name1, [Database_Name2, .....,  
Database_NameN];
```

Using this statement, we have no need to write multiple statements for deleting multiple databases. We can specify all the databases by using a comma in a single statement, as shown in the above syntax.

Examples of Drop Database Statement in SQL

In this article, we took the following two examples that will help how to run and perform the Drop Database query in SQL:

Example1:

Suppose, we want to delete the Student database with all its data from the database system so, firstly we have to check that the Student database exists in the system or not by using the following statement:

```
SHOW DATABASES;
```

If the Student database is shown in the output, then we have to type the following query in SQL for removing the Student database:

```
DROP DATABASE Student;
```

If the Student database does not exist in the database system and we run the above query in SQL, then the query will show the following output:

Can't drop database 'Student'; database doesn't exist

Example2:

Suppose, we want to delete the College database with all its tables and views from the database system, firstly we have to check that if the College database exists in the system or not by using the following statement:

```
SHOW DATABASES;
```

If the College database is shown in the output, then you have to type the following query in SQL for removing the College database permanently:

```
DROP DATABASE College;
```

If the College database does not exist in the database system, and we run the above query in SQL, then this query will show the following output:

Can't drop database 'College'; database doesn't exist

In sqlite3

```
rm class.db
```

SQL Table

Table is a collection of data, organized in terms of rows and columns. In DBMS term, table is known as relation and row as tuple.

Note: A table has a specified number of columns, but can have any number of rows.

SQL TABLE Variable

The **SQL Table variable** is used to create, modify, rename, copy and delete tables. Table variable was introduced by Microsoft.

It was introduced with SQL server 2000 to be an alternative of temporary tables.

It is a variable where we temporary store records and results. This is same like temp table but in the case of temp table we need to explicitly drop it.

SQL CREATE TABLE

Before discussing the command for creating a table, we shall firstly learn what a table is.

What do you mean by Table?

A table is used to store data in a relational database. It is a collection of rows and columns. Each column in a database table represents the attributes and each row represents a set of columns values in the database table. A row of a table is also known as record.

Following is the example of Simple Student_Table:

Row 1	Column1	Column2	Column3
Row 2	Name	Class	Section
Row 3	Rahul	BCA	A
Row 4	Anjali	MCA	B
Row 5	Sunita	BTECH	A

The following rules should be used while designing table:

- Use descriptive name for tables, columns and indexes. **For Example:** a table consisting of information of students of a school should have field names like rollno, name, class etc and the table should be named as student_info etc.
- Every column of the table should have a valid data type.
- Each table must have atleast one column.
- Columns that define null are defined last, to conserve less storage.
- Create a proper documentation of the meaning of every table and its columns.
- Do not use any cryptive codes or numbers in table name.

How to Create Table?

SQL CREATE TABLE statement is used to create table in a database.If you want to create a table, you should name the table and define its column and each column's data type.

To create a table, you must have the CREATE TABLE system privilege. If you want to create a table in another user's schema then you must have the CREATE ANY TABLE system.

Following syntax is used for creating a table.

```
create table [Schema] <table_name"> (  
    <column_name1> <data type> [Default <expression>] [<Column constraints>],  
    (<column_name2> <data type> [Default <expression>] [<Column constraints>],  
    (<column_namen> <data type> [Default <expression>] [<Column constraints>],  
    [<table_constaints>]);
```

In the above syntax,

- **Schema:** This represents the schema name or the user name to which the table belongs.
- **table_name:** It represents the names of the columns to be created in this table.

- **column_name:** It represents the names of the columns to be created in a table.
- **Data_type:** This represents the data type of each column. When a data type is assigned to a column it must have a length and scale, depending on which data type is being specified. The data type of the columns may vary from one database to another. For example, NUMBER is supported in Oracle database for integer value whereas INT is supported in MySQL.
- **Default <expression>:** It stores the value to be assigned to a column if insert statement omits a value for the column. An expression can be fixed value or any SQL function such as sysdate etc.
- **Column constraints:** The constraints are the rules that must be satisfied for the value in the column to be valid. Column constraints are applied only to the individual column.
- **Table constraints:** The constraints that we apply to group of one or more columns. At table level, one can define any constraint except NOT NULL.

sqlite3 class_5.db //To create database

create table students(id INT NOT NULL, name VARCHAR(20) NOT NULL, age INT NOT NULL, address VARCHAR(50), PRIMARY KEY(id)); //To create table

insert into students(id, name, age, address) values(1, 'Ram', 21, 'UP'); //To insert single data

insert into students(id, name, age, address) values(3, 'Lakshman', 19, 'UP'), (2, 'Bharat', 20, 'UP'), (4, 'Satrugan', 28, 'UP'), (0, 'Hanuman', 20, 'Kiskindha'), (5, 'Vibhishan', 23, 'Lanka'); //To insert multiple data

SELECT * from students ; //To see data

Output:

1|Ram|21|UP

3|Lakshman|19|UP

2|Bharat|20|UP

4|Satrugan|28|UP

0|Hanuman|20|Kiskindha

5|Vibhishan|23|Lanka

SQL DROP TABLE

The **DROP** command is used to remove database objects like tables, databases, views, indexes, or stored procedures permanently. When we remove something by using the DROP command, we can not roll back the action. If we want to recover the deleted data, it comes only after backing up the data. The DROP command is used in DDL statements.

The **DROP TABLE** command in SQL is utilized to permanently delete a table definition and all data from the existing table in the database. This command also deletes all the stored data in the table.

Note: When using the DROP command, use it carefully because once the table is deleted, then all the data stored in it is permanently removed and cannot be recovered.

Syntax

DROP TABLE table_name;

Parameters

DROP: This keyword is utilized to remove the objects from the database.

TABLE: It is used to tell which table you want to eliminate from the database.

table_name: Here, we give the name of the table that refers to the table we want to delete from the database.

Example

sqlite> CREATE TABLE actors_actresses (id INT PRIMARY KEY, name VARCHAR(100), gender VARCHAR(10), age INT);

```
sqlite> INSERT INTO actors_actresses (id, name, gender, age) VALUES (1,
'Leonardo DiCaprio', 'Male', 50), (2, 'Scarlett Johansson', 'Female', 40), (3, 'Tom
Hanks', 'Male', 68), (4, 'Emma Watson', 'Female', 35);
sqlite> SELECT * FROM actors_actresses;
1|Leonardo DiCaprio|Male|50
2|Scarlett Johansson|Female|40
3|Tom Hanks|Male|68
4|Emma Watson|Female|35
sqlite> drop table actors_actresses;
sqlite> SELECT * FROM actors_actresses;
Error: no such table: actors_actresses
```

Advantages of DROP TABLE

Simplifying Database Schema: Databases contain many tables, some of which are unused or outdated. We can drop these kinds of tables from the database and helps maintain a clean and manageable database schema.

Faster Operations: If we remove tables from the database then it deletes the data and associated structures which in turn releases disk space that helps improve performance.

Data Security: If, in some situations, tables stored in a database contain sensitive information which is prone to be attacked by the intruder then dropping these tables from the database can prevent further unauthorized access.

Reduced Backup and Restore Time: When unused tables are removed from the database then overall size of the database is also reduced. This reduced size decreases the time required for database backups and restore operations.

Testing and Development: Sometimes, it is required to create temporary tables in a database for testing and development purposes. These temporary tables are dropped when they are no longer required.

Disadvantages of DROP TABLE

Irreversible Data Loss: It is one of the important disadvantages of DROP TABLE because it permanently deletes the table from the database. It is an irreversible action which cannot be rolled back and delete data forever.

Dependency Errors: Some tables are connected to other database objects such as foreign keys, stored procedures, views, triggers, etc. If we drop these kinds of tables then it may cause errors or break the functionality of those dependent objects.

Recovery Challenges: If the table is dropped accidentally then it becomes difficult to recover data and if the dataset is large then the recovery process becomes time-consuming and complex.

Loss of Associated Objects: When we drop a table from the database then all associated objects such as triggers, constraints, indexes, etc., will also be deleted. If we try to recreate the objects then it takes time and a complex process.

Security Risks: The DROP TABLE command can be exploited in SQL injection attacks. An intruder can inject malicious code to drop important tables which leads to data loss and operation disruption.

SQL DELETE TABLE

The DELETE statement is used to delete rows from a table. If you want to remove a specific row from a table you should use WHERE condition.

```
DELETE FROM table_name [WHERE condition];
```

But if you do not specify the WHERE condition it will remove all the rows from the table.

```
DELETE FROM table_name;
```

There are some more terms similar to DELETE statement like as DROP statement and TRUNCATE statement but they are not exactly same there are some differences between them.

Difference between DELETE and TRUNCATE statements

There is a slight difference b/w delete and truncate statement. The **DELETE statement** only deletes the rows from the table based on the condition defined by WHERE clause or delete all the rows from the table when condition is not specified. But it does not free the space containing by the table.

The **TRUNCATE statement**: it is used to delete all the rows from the table **and free the containing space**.

Let's see an "employee" table.

Emp_id	Name	Address	Salary
1	Aryan	Allahabad	22000
2	Shurabhi	Varanasi	13000
3	Pappu	Delhi	24000

Execute the following query to truncate the table:

```
TRUNCATE TABLE employee;
```

Difference b/w DROP and TRUNCATE statements

When you use the drop statement it deletes the table's row together with the table's definition so all the relationships of that table with other tables will no longer be valid.

When you drop a table:

- Table structure will be dropped
- Relationship will be dropped
- Integrity constraints will be dropped
- Access privileges will also be dropped

On the other hand when we **TRUNCATE** a table, the table structure remains the same, so you will not face any of the above problems.

1. DELETE – Remove rows from a table

-- Delete specific rows

```
DELETE FROM table_name WHERE condition;
```

-- Example: delete students older than 21

```
DELETE FROM students WHERE age > 21;
```

-- Delete all rows (table structure remains)

```
DELETE FROM students;
```

- **Effect: Removes data, but the table structure and columns remain.**
- **Supports WHERE clause** for selective deletion.
- **Can be rolled back** if you are inside a transaction.

2. DROP – Remove the table completely

-- Drop the table entirely

```
DROP TABLE table_name;
```

-- Example:

```
DROP TABLE students;
```

- **Effect: Removes the table structure and all data.**
- After this, the table **no longer exists**.
- Cannot rollback unless inside a transaction (SQLite supports limited transaction rollback).

3. TRUNCATE – Not supported in SQLite

- SQLite **does not have the TRUNCATE TABLE command.**
- To “truncate” (delete all rows) in SQLite, you **use DELETE without a WHERE clause:**

```
DELETE FROM table_name;
```

Optional: **Reset AUTOINCREMENT** if your table uses it:

```
DELETE FROM sqlite_sequence WHERE name='table_name';
```

This resets the rowid counter in SQLite for tables that use AUTOINCREMENT.

SQL RENAME TABLE

Creating tables is a common process to store data when working with databases. We assign specific names to them when creating a table. In some situations, database administrators and users want to change the name of the table in the SQL database because they want to give a more relevant name to the table. They decide to change or modify the names of those tables.

Renaming a Table in SQL

In SQL, renaming a table means giving an existing table a new name without changing or modifying its data or structure. It's like changing the name tag on a storage box. The items inside that storage box remain the same while doing this. Both the **RENAME TABLE** and **ALTER TABLE** statements help in changing the name of the table.

Using the RENAME TABLE statement in SQL to rename a table

Syntax:

```
RENAME TABLE old_table_name TO new_table_name;
```

Description:

RENAME TABLE: It is a SQL command used to rename the existing table in a database.

old_table_name: It is the current name of the table that we want to rename.

TO: It is the SQL keyword used to specify a new name.

new_table_name: It is the desired new name for the table.

Examples of using RENAME TABLE in SQL

```
sqlite> SELECT * from actors_actresses ;
```

```
1|Leonardo DiCaprio|Male|50
```

```
sqlite> alter table actors_actresses rename to actors_actresses_12;
```

```
sqlite> SELECT * from actors_actresses_12 ;
```

```
1|Leonardo DiCaprio|Male|50
```

Important points for renaming tables in SQL

Utilizing meaningful and clear names

You should always prefer to rename a table based on the information it stores. The name should directly reflect the content inside that table because this will avoid confusion for developers and make your database more understandable.

Following naming conventions

You should maintain consistent naming rules such as lowercase, underscores or prefixes because this will help us to reduce errors when working with large databases.

Test in a development environment first

The renaming process should first be tested in a development environment before being used on a live database. It ensures that nothing breaks in queries, applications or stored procedures.

Update all references immediately

When you rename a table then the immediate work to do is to update all SQL queries, scripts, views and application code otherwise old references to the table name will cause errors.

Limitations of renaming a table in SQL

Breaks existing queries or code

Renaming a table does not automatically update applications or queries. At any place, using the old table name will fail until it is corrected manually.

Dependencies are not updated automatically

Stored procedures, triggers, views or foreign keys that depend on the old names will not adjust or change themselves. They must be updated by the developer.

Requires proper permissions

The process of renaming a table is not done by every user. If anyone wants to rename a table then they must be the owner of the table or have administrative privileges which can limit flexibility.

Not always supported uniformly across databases

Different SQL databases use different commands as we saw in the above cases. This makes it harder to use a single rename command across multiple systems.